

Jerarquía de excepciones

Armar una jerarquía de excepciones

- **Categorías o tipos de excepciones**
 - **Checked Exceptions:**
 - Heredan de Exception.
 - El compilador obliga a chequear el eventual lanzamiento de la excepción.
 - El compilador obliga a manejar o lanzar la excepción.
 - **Unchecked Exceptions:**
 - Heredan de RuntimeException.
 - No estamos obligados a chequear el eventual lanzamiento de la excepción.
 - No es necesario el tratamiento de la excepción.

Opciones para manejo de Checked Exceptions

//opción 1: manejar y relanzar

```
public void unMetodoQuePuedeFallar() throws OtraCheckedException {  
  
    try {  
  
        //codigo que puede lanzar una checked exception  
  
    } catch(MiCheckedException e) {  
  
        //atrapo, manejo y lanzo una nueva  
  
        throw new OtraCheckedException();  
  
    }  
  
}
```

Opciones para manejo de Checked Exceptions

//opción 2: manejar y pero no relanzar

```
public void unMetodoQuePuedeFallar() {
```

```
    try {
```

```
        //codigo que puede lanzar una checked exception
```

```
    } catch(MiCheckedException e) {
```

```
        //atrapo y manejo
```

```
        this.mailClient.enviarMail("soporte@abc.com", "error!");
```

```
    }
```

```
}
```

Opciones para manejo de Checked Exceptions

//opción 3: lanzar tal cual viene

```
public void unMetodoQuePuedeFallar() throws IOException {
```

```
    //codigo que puede lanzar una checked exception
```

```
    //en este caso, código que puede lanzar IOException
```

```
    //no hago nada, solo aviso al llamador de la eventual
```

```
    //excepción
```

```
}
```


Opciones para manejo de Unchecked Exceptions

Las opciones son exactamente las mismas.

Recordar que en el caso de las unchecked no estamos obligados a poner el try ni el catch, pero si lo queremos hacer, podemos usar cualquiera de las tres opciones.

De lo contrario, todo se reduce a:

```
public void unMetodoQuePuedeFallar() {  
  
    //codigo que puede lanzar una UncheckedException  
  
}
```

Como no estoy obligado a chequear ni manejar, puede seguir todo como si no hubiera problemas.

¿Cuál es la mejor?

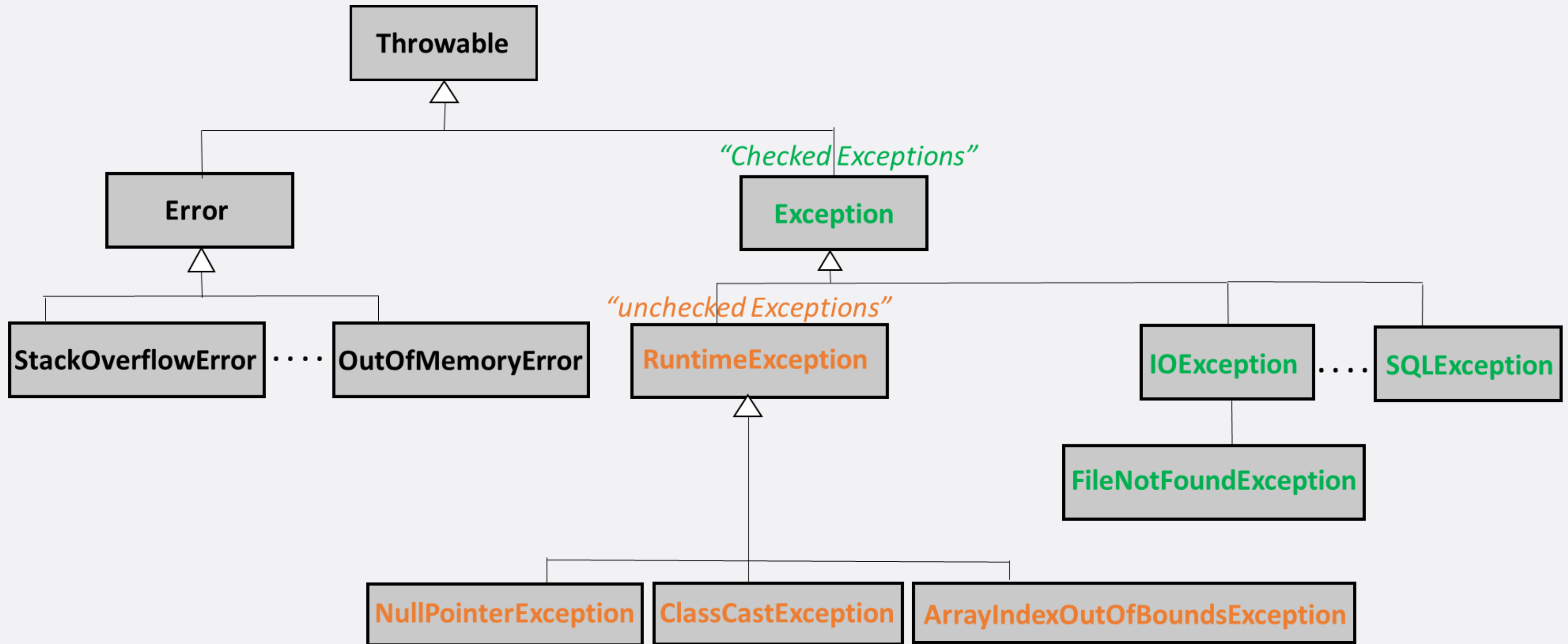
Todo depende de:

- El negocio.
- El problema que queremos resolver.
- El grado de control que queremos tener sobre las eventualidades en nuestro código.

¿Y qué pasa si pueden lanzarse más de una excepción?

- Las opciones son las mismas y, nuevamente, volvemos a depender de lo mismo.
- Pero cada punto de los anteriores nos puede llevar a tener que decidir si manejamos:
 - Manejar todas las excepciones en cada llamada.
 - Realizar una jerarquía y manejar la excepción más genérica.
 - Un mix entre los dos.

Ya conocemos la jerarquía base



Entonces, ¿por qué no aplicamos algo similar?

- ¿Cómo crear nuestras propias excepciones?
 - Extender Exception o RuntimeException
 - `public class BebidaException extends Exception { .... }`
 - `public class SinGasException extends BebidaException{ .... }`
 - `public class TemperaturaException extends BebidaException { ....}`
- Sobrecribir los 4 constructores más comunes:
 - No es obligatorio, porque siempre está el constructor por default.

Constructores posibles

```
public BebidaException() { //constructor por default  
}
```

```
public BebidaException(String message) {  
    super(message); //creo una excepción con un mensaje  
}
```

```
public BebidaException(Throwable cause) {  
    //creo una excepción e indico qué la provocó por eso  
    //por eso el argumento es un Throwable  
    super(cause);  
}
```

```
public BebidaException(String message, Throwable cause) {  
    //el mensaje de esta excepción y qué la provocó  
    super(message, cause);  
}
```

Podemos hacer lo mismo para las hijas de BebidaException.

Respecto del manejo

Se puede avisar que se lanzan todas las excepciones.

```
public void consumirBebida() throws SinGasException, TemperaturaException {  
    if(this.gas < 100) {  
        throw new SinGasException();  
    }  
    if(this.temperatura > 20) {  
        throw new TemperaturaException();  
    }  
}
```

Respecto del manejo

O se puede avisar que se lanza la más genérica.

```
public void consumirBebida() throws BebidaException {  
    if(this.gas < 100) {  
        throw new SinGasException();//SinGasException es una  
BebidaException  
    }  
    if(this.temperatura > 20) {  
        throw new TemperaturaException();//TemperaturaException es una  
BebidaException  
    }  
}
```

Conclusiones

- Las excepciones son clases como cualquier otra.
- Extender de `Exception` o de `RuntimeException` solo convierte a nuestra excepción en Checked o Unchecked:
 - El tratamiento es igual: try – catch – finally
 - Cambia la obligatoriedad del tratamiento:
 - En las checked debo manejar y/o avisar que se lanza la excepción.
 - En las unchecked no es necesario (ni obligatorio).
- Elegir la forma del tratamiento depende del problema a resolver.
- Podemos manejar las excepciones lanzadas:
 - Excepción por excepción (siendo bien específicos en el tratamiento de cada una).
 - Manejar la excepción Padre (siendo más genéricos en el tratamiento, tratando a todas por igual).
 - Combinar una y otra forma según el caso.



© **Universidad de Palermo**

Prohibida la reproducción total o parcial de imágenes y textos.